

Lecture 23b: Generative Adversarial Networks

Two networks, one game

Parts of this chapter adapt LMU I2DL (slds-lmu), CC BY 4.0.

A different bet entirely

L23 ended with a working generative model and an honest complaint: the VAE's samples are **blurry**. The cause was structural - we maximized a likelihood bound, and an averaged prediction scores well on it.

The GAN's bet: stop measuring likelihood altogether. Instead, train a *second* network to tell real images from generated ones - and use its verdict as the loss.

If a critic cannot distinguish your output from real data, your output is indistinguishable from real data. **That is the whole idea**, and it is a remarkably direct answer to "what makes an image look real?"

Generative Adversarial Networks (Goodfellow et al., 2014).

Outline

The adversarial idea

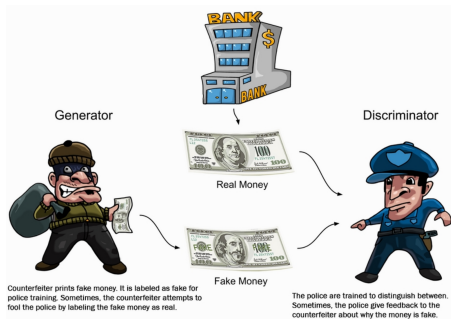
What is actually being optimized

Why this is hard: our own run

The adversarial idea

Replace the loss function with an opponent.

The forger and the inspector



LMU I2DL, CC BY 4.0

Generator G : takes a random vector z and produces an image. It never sees the training data. It only ever learns from the critic's reaction.

Discriminator D : takes an image and outputs one number - the probability that this image is real.

They are trained **alternately** and want opposite things. D improves at spotting fakes; G improves at fooling D . Each one's progress is the other's problem.

Note what is absent: no reconstruction term, no likelihood, no encoder. G is judged only on whether its output passes.

One training step, concretely

Update D : show it a batch of real images (label 1) and a batch of generated ones (label 0). Plain binary classification.

Update G : generate a fresh batch, push it through D , and change G so that D calls them *real*.

The gradient reaching G flows **through** D . That is why D must be trained too, and why a D that is too good or too weak both break learning.

Algorithm 1 Minibatch stochastic gradient descent training of generative adversarial nets. The number of steps to apply to the discriminator, k , is a hyperparameter. We used $k = 1$, the least expensive option, in our experiments.

for number of training iterations **do**

for k steps **do**

- Sample minibatch of m noise samples $\{\mathbf{z}^{(1)}, \dots, \mathbf{z}^{(m)}\}$ from noise prior $p_g(\mathbf{z})$.
- Sample minibatch of m examples $\{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}\}$ from data generating distribution $p_{\text{data}}(\mathbf{x})$.
- Update the discriminator by ascending its stochastic gradient:

$$\nabla_{\theta_d} \frac{1}{m} \sum_{i=1}^m \left[\log D(\mathbf{x}^{(i)}) + \log (1 - D(G(\mathbf{z}^{(i)}))) \right].$$

end for

- Sample minibatch of m noise samples $\{\mathbf{z}^{(1)}, \dots, \mathbf{z}^{(m)}\}$ from noise prior $p_g(\mathbf{z})$.
- Update the generator by descending its stochastic gradient:

$$\nabla_{\theta_g} \frac{1}{m} \sum_{i=1}^m \log (1 - D(G(\mathbf{z}^{(i)}))).$$

end for

The gradient-based updates can use any standard gradient-based learning rule. We used momentum in our experiments.

LMU I2DL, CC BY 4.0

What is actually being optimized

A game has no loss function. It has an equilibrium.

The minimax objective

$$\min_G \max_D \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}} [\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim p(\mathbf{z})} [\log (1 - D(G(\mathbf{z})))]$$

Read it in two halves. D wants the first term large (call real images real) and the second large (call fakes fake). G can only touch the second term, and wants it *small*.

Notice what this is not. There is no single quantity being minimized. Two players move in opposite directions on the same number.

Everything difficult about GANs follows from that one fact.

If the discriminator were perfect

Hold G fixed and solve for the best possible D . It is a calculus exercise, and the answer is clean:

$$D^*(\mathbf{x}) = \frac{p_{\text{data}}(\mathbf{x})}{p_{\text{data}}(\mathbf{x}) + p_G(\mathbf{x})}$$

Sanity-check it. Where only real data lives, $p_G = 0$ and $D^* = 1$. Where only fakes live, $D^* = 0$.

Where the two distributions agree, $D^* = \frac{1}{2}$ - the critic is reduced to guessing.

Substituting D^* back in, the generator turns out to be minimizing the **Jensen-Shannon divergence** between p_{data} and p_G - a symmetric cousin of the KL divergence from L23.

So GANs do minimize a divergence after all - just never one you can evaluate. The loss you watch during training is not it.

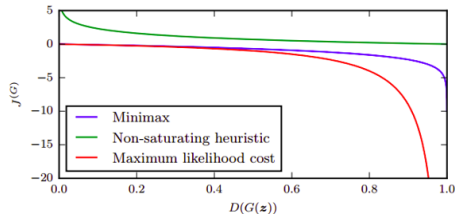
And the theory quietly assumes too much: that D is optimal at every step, and that G can represent any distribution. Neither holds. The equilibrium argument is a guide, not a guarantee.

The first thing that breaks

Early in training G is terrible, so D rejects everything and $D(G(\mathbf{z})) \approx 0$.

Look at what happens to G 's gradient there: $\log(1 - D(G(\mathbf{z})))$ is **flat** near 0. The generator is losing badly and learning nothing - exactly when it most needs signal.

The fix (same paper): have G *maximize* $\log D(G(\mathbf{z}))$ instead of minimizing $\log(1 - D(G(\mathbf{z})))$. Same fixed point, steep gradient where it matters. This is the **non-saturating** loss, and essentially every implementation uses it.



LMU I2DL, CC BY 4.0

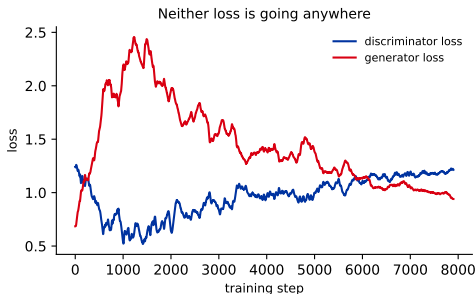
Our own implementation uses it too - see the comment in `py_src/gan_digits.py`.

Why this is hard: our own run

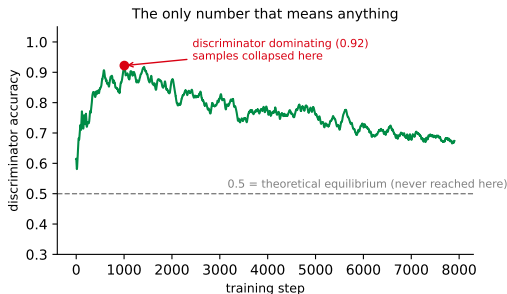
Everything from here is measured on a GAN trained for these slides.

The loss tells you nothing

A GAN on 8x8 handwritten digits - the same dataset a later chapter will reuse, so the two families can be compared on identical data - 8000 steps on a laptop:



Left: the two losses wander and cross. Neither descends. If you are used to watching a training curve to decide when to stop, that instinct is now useless.



Right: discriminator accuracy is the one readable signal. Theory says it should settle at 0.5. Ours peaks at **0.92**, then drifts down to 0.67 - and never reaches equilibrium.

Mode collapse, as it happens

The same ten latent vectors $\mathbf{z}$, decoded at four points in training:

Same 10 latent vectors, followed through training

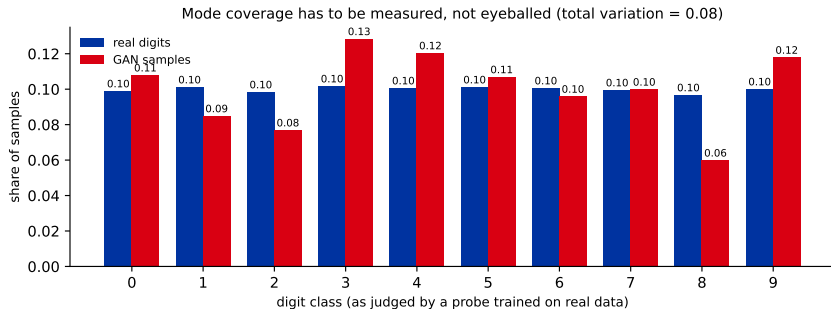


Look at step 500. Ten *different* inputs produce essentially the *same* output. The generator has found one image that fools D and stopped exploring - textbook **mode collapse**. By step 2000 it has recovered to roughly two recurring shapes - better, still far short of ten digits. By 8000 it has escaped into real variety.

Compare with the accuracy curve: the collapse sits exactly where D was dominating at 0.92. **An overpowering critic causes collapse.**

Mode collapse you cannot see

Escaping visible collapse is not the same as covering the data. Train a plain classifier on the *real* digits (accuracy 0.996), then use it to label 2000 generated ones:



Overall coverage is decent - total variation 0.079.
This run did **not** catastrophically collapse, and I am not going to pretend it did.

But look at digit **8**: 0.060 generated against 0.097 real. Roughly **38% of the 8s are missing**, and no amount of staring at the samples would have told you.

The failure modes, named

Mode collapse. G maps many z to few outputs. Sharp images, no diversity. Ours did it at step 500 and partly recovered.

Non-convergence. The pair oscillates instead of settling. G fixes what D just learned, D learns the new tell, forever.

Vanishing gradient. D wins outright, G gets no signal. The non-saturating loss helps but does not eliminate it.

The simplest possible example. Take the toy game $V(x, y) = xy$, where one player lowers V by moving x and the other raises it by moving y . The unique equilibrium is $(0, 0)$.

Follow gradient steps and you get this: five states, and state 5 is state 1 again.

• State 1:	<table><tr><td>$x > 0$</td><td>$y > 0$</td><td>$V > 0$</td></tr></table>	$x > 0$	$y > 0$	$V > 0$	<table><tr><td>Increase y</td><td>Decrease x</td></tr></table>	Increase y	Decrease x
$x > 0$	$y > 0$	$V > 0$					
Increase y	Decrease x						
• State 2:	<table><tr><td>$x < 0$</td><td>$y > 0$</td><td>$V < 0$</td></tr></table>	$x < 0$	$y > 0$	$V < 0$	<table><tr><td>Decrease y</td><td>Decrease x</td></tr></table>	Decrease y	Decrease x
$x < 0$	$y > 0$	$V < 0$					
Decrease y	Decrease x						
• State 3:	<table><tr><td>$x < 0$</td><td>$y < 0$</td><td>$V > 0$</td></tr></table>	$x < 0$	$y < 0$	$V > 0$	<table><tr><td>Decrease y</td><td>Increase x</td></tr></table>	Decrease y	Increase x
$x < 0$	$y < 0$	$V > 0$					
Decrease y	Increase x						
• State 4 :	<table><tr><td>$x > 0$</td><td>$y < 0$</td><td>$V < 0$</td></tr></table>	$x > 0$	$y < 0$	$V < 0$	<table><tr><td>Increase y</td><td>Increase x</td></tr></table>	Increase y	Increase x
$x > 0$	$y < 0$	$V < 0$					
Increase y	Increase x						
• State 5:	<table><tr><td>$x > 0$</td><td>$y > 0$</td><td>$V > 0$</td></tr></table> == State 1	$x > 0$	$y > 0$	$V > 0$	<table><tr><td>Increase y</td><td>Decrease x</td></tr></table>	Increase y	Decrease x
$x > 0$	$y > 0$	$V > 0$					
Increase y	Decrease x						

LMU I2DL, CC BY 4.0

It **orbits the solution forever** - in a game with two variables and no networks at all. All three failures share one root: **no fixed objective to descend**.

What actually makes it train

A decade of empirical folklore, most of it from **DCGAN - Deep Convolutional GAN** (Radford et al., 2015):

- ▶ strided convolutions instead of pooling, both directions
- ▶ batch normalization in both networks
- ▶ **LeakyReLU** in D - a dead ReLU kills the gradient G depends on
- ▶ Adam with $\beta_1 = 0.5$, not the usual 0.9
- ▶ balance the two updates; never let one race ahead

Our run uses the LeakyReLU, dropout and $\beta_1 = 0.5$ items. They are not optional decoration - $\beta_1 = 0.9$ diverges.

This list is the tell. A method that needs five architectural superstitions to converge is a method fighting its own formulation.

Later work attacked the root instead - Wasserstein GAN (Arjovsky et al., 2017) replaces the divergence; spectral normalization constrains D .

Recap

1. A GAN replaces the loss function with a second network.
2. The objective is a **minimax game**, not a minimization - there is no number that going down means progress.
3. With an optimal D , G minimizes the Jensen-Shannon divergence - which you can never evaluate.
4. The naive generator loss saturates; everyone uses the non-saturating form.
5. Our own run: losses that wander, D accuracy peaking at 0.92, visible mode collapse at step 500, and 38% of the 8s still missing at the end.
6. Training is stabilized by a list of tricks, which is itself a diagnosis.

Next: everything above is the unconditional case - a GAN that draws *something*.

L23c is what people actually built with these: asking for a specific class, turning sketches into photographs, editing faces by moving in latent space - and how you evaluate any of it when likelihood is unavailable.